

SELF-REFLECTIVE REPORT

Object-Oriented Programming Fundamentals in Java

Student Name	BEN NASER MHAMED JAMAL ALI
Student ID	202409010643
Passport Number	AB732634
Class Code	4007
Program	Software Engineering
Course	BIT1123 Object Oriented Programming
Institution	City University Malaysia - Cyberjaya Campus

Assignment 1 - Individual (20%)

1. Introduction

This report reflects on my learning journey through the official Week 1-10 Java lab tutorials for BIT1123 Object Oriented Programming. The tutorial sequence began with GitHub Codespaces and basic Java programs, then progressed through classes, constructors, inheritance, method overriding, encapsulation, abstraction, collections, file handling, exception handling, and a Swing graphical user interface. Organising the work in one GitHub-ready repository also helped me understand how source code, documentation, and version control contribute to professional software development.

At the beginning, my main focus was making individual statements work correctly. As the tutorials progressed, I started to think more carefully about how a program should be divided into objects with clear responsibilities. The most important improvement was learning to connect theory with code: a class became a practical blueprint, encapsulation became protection for object state, and polymorphism became a way to write flexible code that can work with multiple related object types.

Learning objectives

Objective	Evidence in repository
Use Java syntax confidently	Hello World, grade calculation, input, conditions, and loops
Model problems with objects	Student, university people, employees, lecturers, and appliances
Apply core OOP principles	Constructors, encapsulation, inheritance, overriding, and abstraction
Store and recover data	ArrayList, buffered file writing, file reading, and exception handling
Build a Java interface	Event-driven Swing Programming Quiz Battle GUI
Document and present work	Consistent folders, tutorial briefs, weekly notes, and main README

2. Knowledge Gained from the Tutorials

Week 1 - GitHub Codespaces and Java basics

I learned how to create the subject repository, use a consistent naming convention, work in GitHub Codespaces, and follow the basic Git workflow. Completing **HelloWorld.java** and **StudentGrade.java** also reinforced the structure of a Java program, the **main** method, console input, conditional logic, and output.

Week 2 - Classes, objects, and constructors

I created a Student class with name, age, and GPA attributes. A constructor initialised each new object, while `displayInfo`, `study`, and `takeExam` represented the student's behaviour. This tutorial clarified the difference between a class as a blueprint and an object as a specific instance.

Weeks 3-4 - Inheritance and method overriding

I modelled Student and Lecturer as specialised versions of Person. Both subclasses reuse the parent's name and ID through **super**, then override **introduce()**. Calling the method through Person references demonstrated that the actual object decides which version runs.

Week 5 - Encapsulation

I built a Student Information System in which student ID, name, CGPA, and programme are private. Getters and setters provide controlled access, while the CGPA setter checks that the value is within the valid range. The written questions helped me explain why public fields weaken data protection.

Week 6 - Employee and Lecturer inheritance

I created Employee as a base class containing ID and name. Lecturer extends it with subject and department information, using **super** to initialise the inherited part. This activity showed how inheritance reduces duplication while allowing a specialised class to add its own data and behaviour.

Week 7 - Abstraction with smart-home appliances

I used an abstract Appliance class to provide shared brand, power-on, and power-off behaviour while requiring every appliance to implement **operate()**. Washing machine, refrigerator, air conditioner, and microwave objects each perform that operation differently. This showed how abstraction hides details behind a common design.

Weeks 8-9 - ArrayList and file handling

I developed a to-do application that accepts three tasks into an **ArrayList**, displays them, saves them to **task.txt**, and loads them again. `BufferedWriter`, `BufferedReader`, `try-with-resources`, loops, and `IOException` handling showed how a program can preserve information after it closes.

Week 10 - Swing Programming Quiz Battle

I completed a desktop quiz using `JFrame`, `JLabel`, `JButton`, `ActionListener`, and `Questions` objects. Button-click events check the selected answer, update the score and feedback, and move to the next question. This was my first clear example of event-driven programming and connected OOP design with a visual interface.

3. Challenges Encountered

One challenge was moving from procedural thinking to object-oriented design. It was initially easier to place all variables and operations inside the main method. However, this quickly became difficult to read and maintain. Deciding which class should own a field or method required more planning than writing a short sequence of statements.

Input and persistence were another challenge. Scanner input must be validated, and file operations can fail because a path is unavailable or a file cannot be opened. I also had to distinguish compile-time errors, runtime errors, and logical errors. A program can compile successfully but still behave incorrectly when a value is invalid or file resources are not closed properly.

Inheritance and overriding were conceptually difficult because they involve both declared types and actual object types. I needed to understand why a Person reference can refer to a Student or Lecturer and why the overridden introduction runs. Swing introduced a different challenge because the program responds to button events rather than following only a simple top-to-bottom console flow.

4. How the Challenges Were Overcome

Challenge	Approach used
Large main methods	Separated data and behaviour into small classes with one clear responsibility.
Invalid student data	Kept fields private and validated CGPA through a setter.
Understanding inheritance	Started with Person and Employee fields, then added specialised subclasses.
Understanding overriding	Compared introduce() through Person, Student, and Lecturer objects.
File handling	Used try-with-resources and caught IOException with readable messages.
Swing event flow	Separated question data, GUI display, and button-event handling into clear methods.
Debugging	Read error messages from the first relevant line and tested one change at a time.

Breaking each problem into smaller steps was the most effective method. I first identified the required data, then the operations, and finally the relationships between classes. I tested normal cases as well as invalid or boundary cases. Keeping each weekly example independent also made errors easier to isolate and allowed me to compare my understanding across the semester.

5. Improvements in Java Programming Skills

My code has become more structured and intentional. I can now choose meaningful names, place repeated behaviour in methods, use constructors to initialise objects, and restrict direct access to internal data. I have also improved at selecting appropriate tools from the Java standard library, particularly Scanner, ArrayList, buffered file classes, and Swing components.

I now think about failure conditions before running a program. Instead of assuming that data or file operations will always succeed, I validate CGPA values and catch IOException when saving or loading tasks. In the GUI, I also manage the end of the question list so that buttons cannot access an invalid array position.

Documentation and organisation also improved. The repository uses a consistent folder convention, a README for every grouped tutorial, clear run commands, and a main README that explains the complete learning progression. These practices make the work easier for another developer or lecturer to review.

6. Understanding of OOP Concepts

Concept	My understanding
Encapsulation	Keep object data private and provide controlled operations that protect valid state.
Inheritance	Create a specialised class from a genuine shared base when an is-a relationship exists.
Polymorphism	Use a common parent or interface type while allowing each concrete object to respond differently.
Abstraction	Expose essential operations while hiding implementation details that users of a class do not need.

These concepts work together. Encapsulation protects student data, inheritance shares a suitable Person or Employee foundation, overriding specialises behaviour, and abstraction gives every appliance a common design while hiding its implementation. The Swing quiz also demonstrates composition because the GUI contains and uses Questions objects rather than inheriting from them.

7. Future Learning Plans

1. Practise unit testing with JUnit so that expected behaviour can be checked automatically.
2. Learn Java generics, streams, lambda expressions, and more advanced collection operations.
3. Study SOLID principles and common design patterns to improve class design.
4. Connect Java applications to databases using JDBC and later use the Spring Boot framework.
5. Continue using GitHub with meaningful commits, branches, issues, and project documentation.
6. Build a larger portfolio project with authentication, persistent data, validation, and automated tests.

8. Conclusion

The tutorial journey provided a practical foundation in Java and object-oriented programming. I progressed from GitHub and basic console programs to designing related classes, saving data in a file, and building an event-driven Swing application. The most valuable lesson was that good programming is not only producing the correct output; it is also creating code that is understandable, safe, reusable, and easy to extend.

The challenges with input, object relationships, dynamic method behaviour, file handling, and GUI events strengthened my problem-solving process. By organising the completed work in one documented repository, I also developed a clearer understanding of professional presentation and version control. I plan to build on this foundation through testing, database development, frameworks, and larger software projects.

9. GitHub Repository URL

<https://github.com/mbennaser288-spec/tutorials>